Research Proposal: JADA-GRPO

Jacobian-Aware Debiased Adaptive Continuation Allocation for GRPO-MA

Version: 0.2
Date: 2026-09-30

---

1. Executive Summary

GRPO and related reinforcement-learning methods for language models estimate relative advantage from groups of sampled responses. GRPO-MA extends this idea by generating multiple reasoning trajectories and multiple answer continuations per trajectory so that thought-level credit can be estimated more reliably.

A typical GRPO-MA rollout with $K = 4$ thoughts and $M = 4$ answer continuations per thought is:

```
Prompt
├── Thought T1
│   ├── Answer A11
│   ├── Answer A12
│   ├── Answer A13
│   └── Answer A14
├── Thought T2
│   ├── Answer A21
│   ├── Answer A22
│   ├── Answer A23
│   └── Answer A24
├── Thought T3
│   └── 4 answers
└── Thought T4
    └── 4 answers
```

The standard design allocates the same number of continuations to every thought:

$$M_1 = M_2 = \cdots = M_K = M.$$

However, different thoughts can have substantially different uncertainty. Some may be almost certainly correct or incorrect, while others may lie near the decision boundary and require substantially more samples for stable estimation.

This proposal introduces **JADA-GRPO**:

> **Jacobian-Aware Debiased Adaptive Continuation Allocation for GRPO-MA**

JADA-GRPO has three tightly connected components:

1. **Group-normalization-aware uncertainty estimation.** Allocation is based not only on reward variance, but also on how uncertainty in one thought affects the entire vector of group-normalized thought advantages.
2. **Adaptive continuation allocation.** The remaining rollout budget is assigned to thoughts with the largest expected marginal reduction in advantage-estimation error.
3. **Bias-controlled pilot reuse.** Pilot samples used for allocation are reused in the final value estimate through a precommitted combination rule that avoids direct adaptive-selection bias at the thought-value level.

The central information score is:

$$\boxed{c_j = \sigma_j^2 \|J_{:j}\|_2^2}$$

where σ_j^2 is the reward uncertainty of thought j, and $J_{:j}$ is the j-th column of the Jacobian of the group-normalized advantage vector with respect to thought values.

The corresponding continuous allocation rule is:

$$\boxed{M_j^* \propto \sigma_j \|J_{:j}\|_2}$$

The main hypothesis is that, under a fixed continuation budget, this allocation produces more accurate thought-level credit estimates than uniform or variance-only allocation, and that this reduction in advantage-estimation error reduces policy-gradient estimation error and improves performance per unit of generation compute.

---

2. Background

2.1. Reinforcement Learning for Language Models

For a language model:

- **state**: the prompt and previously generated tokens;
- **action**: the next generated token;
- **policy**: the language model;
- **trajectory**: the full generated response or reasoning trace;
- **reward**: a scalar score assigned to the generated output.

Example:

```
Prompt:
What is 13 × 7?

Response:
13 × 7 = 91

Reward = 1
```

while:

```
Response:
13 × 7 = 81

Reward = 0
```

The training objective is to increase the probability of trajectories associated with higher reward.

---

3. Policy Gradient and Advantage

A simplified policy-gradient estimator can be written as:

$$g = A \nabla_\theta \log \pi_\theta(y|x),$$

where θ denotes model parameters, $\pi_\theta(y|x)$ is the probability of response y, and A is the advantage associated with that response.

Intuitively:

```
A > 0
→ response is better than the reference level
→ increase its probability

A < 0
→ response is worse than the reference level
→ decrease its probability
```

Therefore, inaccurate advantage estimates directly affect the direction and magnitude of the policy update.

---

4. GRPO

GRPO generates a group of responses for the same prompt.

```
Prompt
├── R1 → reward 1
├── R2 → reward 1
├── R3 → reward 0
└── R4 → reward 0
```

The group mean reward is:

$$\bar{r} = \frac{1+1+0+0}{4} = 0.5.$$

A simplified group-normalized advantage is:

$$A_i = \frac{r_i - \bar{r}}{s_r + \epsilon}.$$

The important property is that the advantage of one response depends not only on its own reward, but also on the reward distribution of the entire group.

---

5. GRPO-MA

GRPO-MA introduces a two-level structure:

1. generate multiple reasoning trajectories or thoughts;
2. generate multiple answer continuations from each thought.

Example:

```
Prompt
│
├── Thought T1
│    ├── reward 1
│    ├── reward 1
│    ├── reward 1
│    └── reward 1
```

```
│
├── Thought T2
│    ├── reward 1
│    ├── reward 0
│    ├── reward 1
│    └── reward 0
│
├── Thought T3
│    ├── reward 0
│    ├── reward 1
│    ├── reward 0
│    └── reward 1
│
└── Thought T4
     ├── reward 0
     ├── reward 0
     ├── reward 0
     └── reward 0
```

The value of thought j is estimated by:

$$\hat{\mu}_j = \frac{1}{M} \sum_{m=1}^{M} R_{jm}.$$

For the example above:

$$\hat{\mu} = [1.0,\ 0.5,\ 0.5,\ 0.0].$$

The thought-level advantage is then computed by group normalization:

$$A_j = \frac{\hat{\mu}_j - \bar{\hat{\mu}}}{s_{\hat{\mu}} + \epsilon}.$$

Increasing the number of answer continuations reduces the estimation noise of $\hat{\mu}_j$, but the standard setting allocates the same number of samples to all thoughts.

---

6. Research Gap

6.1. Uniform Continuation Allocation

GRPO-MA typically uses:

$$M_1 = M_2 = \cdots = M_K.$$

However:

$$\mathrm{Var}(R|T_1) \neq \mathrm{Var}(R|T_2)$$

in general.

Consider true success probabilities:

$$p = [0.95,\ 0.55,\ 0.45,\ 0.05].$$

For binary reward:

$$\sigma_j^2 = p_j(1 - p_j).$$

Therefore:

$$\sigma^2 = [0.0475,\ 0.2475,\ 0.2475,\ 0.0475].$$

Thoughts T_2 and T_3 are much harder to estimate than T_1 and T_4, suggesting that uniform allocation may waste part of the rollout budget.

6.2. Reward Variance Does Not Capture Group Coupling

A natural adaptive baseline is:

$$M_j \propto \sigma_j.$$

However, thought advantage is:

$$A_i = \frac{\mu_i - \bar{\mu}}{s_\mu + \epsilon}.$$

Changing μ_j changes the value of thought j, the group mean, the group standard deviation, and potentially every normalized advantage A_i. Therefore, local reward variance alone does not measure the global impact of estimating a thought more accurately.

The key question is:

> How strongly does uncertainty in thought j affect the entire group-normalized advantage vector?

6.3. Pilot Sampling Creates Overhead

Adaptive allocation requires an initial uncertainty estimate.

```
2 pilot answers per thought
× 4 thoughts
= 8 pilot answers
```

With a total budget of 16 answers:

```
8 pilot answers
+
8 adaptively allocated main answers
```

If the pilot answers are discarded after allocation, only half of the generated data contributes directly to the final estimator.

6.4. Naive Pilot Reuse Can Introduce Adaptive-Selection Bias

Suppose the pilot outcomes for T_2 are:

```
[1, 0]
```

The allocator interprets this as high uncertainty and assigns more main samples to T_2. If the pilot samples are then reused with a sample-size-dependent weight, the pilot observations influence both the allocation and their own final weight, creating an adaptive-selection issue.

---

7. Research Questions

RQ1

Does Jacobian-aware continuation allocation estimate thought advantages more accurately than uniform and variance-only allocation under the same rollout budget?

RQ2

Can pilot samples be reused without introducing direct adaptive-selection bias in the thought-value estimator?

RQ3

Does lower thought-advantage estimation error lead to lower policy-gradient estimation error?

RQ4

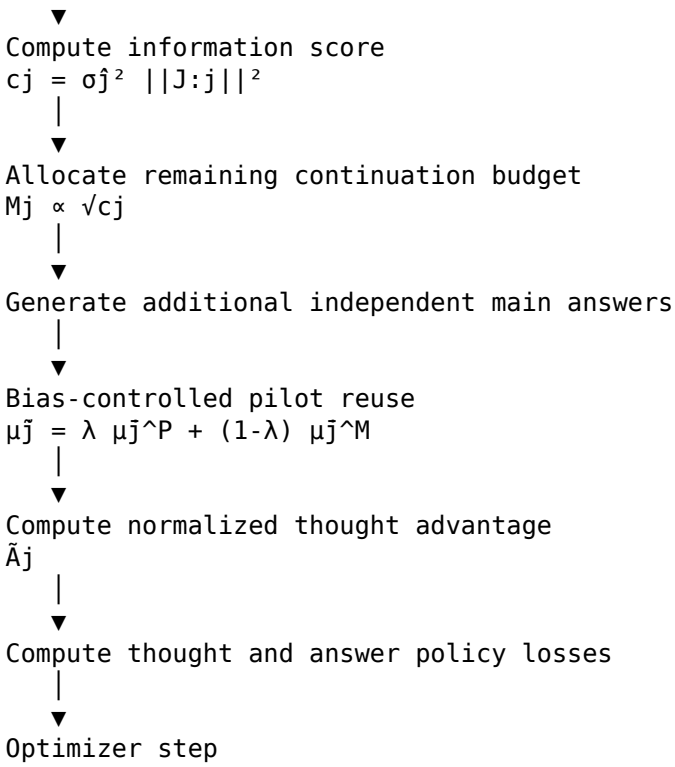
Does improved credit estimation translate into better performance per unit of generation compute during online RL training?

---

8. Proposed Method: JADA-GRPO

The full pipeline is:

```
Prompt x
   │
   ▼
Generate K thoughts
T1, ..., TK
   │
   ▼
Fixed pilot sampling
m0 answers per thought
   │
   ▼
Estimate pilot statistics
μĵ^P, σĵ²
   │
   ▼
Compute group-normalization Jacobian
Jij = ∂Ai / ∂μj
   │
```



A critical implementation constraint is:

> The policy must not be updated between pilot generation and main generation within the same rollout batch.

---

9. Pilot Sampling and Stabilized Statistics

For each thought, generate m_0 pilot answer continuations.

Example:

$$K = 4, \qquad m_0 = 2.$$

Pilot rewards:

```
T1 → [1, 1]
T2 → [1, 0]
T3 → [0, 1]
T4 → [0, 0]
```

The raw means are:

$$[1,\ 0.5,\ 0.5,\ 0].$$

With only two observations per thought, direct empirical estimates are unstable. For binary reward, use a Beta prior:

$$p_j \sim \text{Beta}(\alpha_0, \beta_0),$$

with the default:

$$\alpha_0 = \beta_0 = 0.5.$$

If thought j has s_j successes and f_j failures, the posterior is:

$$p_j|\mathcal{D} \sim \text{Beta}(\alpha_0 + s_j, \beta_0 + f_j).$$

The posterior mean is:

$$\hat{\mu}_j = \frac{\alpha_j}{\alpha_j + \beta_j}.$$

A posterior predictive Bernoulli variance proxy is:

$$\hat{\sigma}_j^2 = E[p_j(1-p_j)|\mathcal{D}] = \frac{\alpha_j \beta_j}{(\alpha_j + \beta_j)(\alpha_j + \beta_j + 1)}.$$

For the example above:

$$\hat{\mu} = [0.833,\ 0.5,\ 0.5,\ 0.167].$$

This avoids concluding that uncertainty is zero simply because both pilot samples happened to agree.

---

10. Group-Normalized Thought Advantage

Let:

$$\mu = [\mu_1, \ldots, \mu_K]^T.$$

The group mean is:

$$\bar{\mu} = \frac{1}{K} \sum_i \mu_i.$$

Define:

$$v_\mu = \frac{1}{K} \sum_i (\mu_i - \bar{\mu})^2$$

and:

$$s_\mu = \sqrt{v_\mu + \epsilon}.$$

The normalized thought advantage is:

$$A_i = \frac{\mu_i - \bar{\mu}}{s_\mu}.$$

In vector form:

$$\mathbf{A} = f(\mu).$$

---

11. Group-Normalization Jacobian

Define:

$$J_{ij} = \frac{\partial A_i}{\partial \mu_j}.$$

The column $J_{:j}$ measures how strongly uncertainty in μ_j affects the entire normalized advantage vector.

Suppose:

$$\mu = [0.90,\ 0.52,\ 0.48,\ 0.10].$$

If:

$$\mu_2 : 0.52 \to 0.62,$$

then A_2 changes directly, but the group mean and standard deviation also change. Therefore, A_1, A_3, and A_4 may change as well.

The first implementation should use automatic differentiation:

```
import torch

def group_advantage(mu, eps=1e-6):
    mean = mu.mean()
    var = ((mu - mean) ** 2).mean()
    std = torch.sqrt(var + eps)
    return (mu - mean) / std

mu = torch.tensor(
    [0.83, 0.50, 0.50, 0.17],
    dtype=torch.float64,
    requires_grad=True,
)

J = torch.autograd.functional.jacobian(
    group_advantage,
    mu,
)
```

The Jacobian must be validated with finite differences:

$$\frac{\partial A_i}{\partial \mu_j} \approx \frac{A_i(\mu + \delta e_j) - A_i(\mu - \delta e_j)}{2\delta}.$$

---

12. Advantage-Error Approximation

Assume:

$$\widehat{\mu}_j = \mu_j + \delta_j.$$

A first-order Taylor approximation gives:

$$\widehat{\mathbf{A}} - \mathbf{A} \approx J\delta.$$

Assume the estimation errors across thoughts are approximately independent and:

$$\mathrm{Var}(\widehat{\mu}_j) \approx \frac{\sigma_j^2}{M_j}.$$

Then:

$$E\left[\|\widehat{\mathbf{A}} - \mathbf{A}\|_2^2\right] \approx \mathrm{tr}(J\Sigma_\mu J^T),$$

where:

$$\Sigma_\mu = \mathrm{diag}\left(\frac{\sigma_1^2}{M_1}, \ldots, \frac{\sigma_K^2}{M_K}\right).$$

Therefore:

$$\boxed{\mathcal{E}_A \approx \sum_{j=1}^{K} \frac{\sigma_j^2 \|J_{:j}\|_2^2}{M_j}}$$

Define:

$$\boxed{c_j = \sigma_j^2 \|J_{:j}\|_2^2.}$$

This quantity combines local uncertainty and global influence.

---

13. Interpretation of the Information Score

$$c_j = \underbrace{\sigma_j^2}_{\text{uncertainty}} \times \underbrace{\|J_{:j}\|^2}_{\text{group influence}}.$$

Example:

Thought	Reward variance	Jacobian influence	c_j
T_1	0.08	1.0	0.08
T_2	0.20	3.0	0.60
T_3	0.20	2.8	0.56

Thought	Reward variance	Jacobian influence	c_j
T_4	0.08	1.1	0.088

A variance-only allocator sees:

`T2 ≈ T3 > T1 ≈ T4`

JADA-GRPO additionally captures how strongly each thought affects the normalized group credit.

---

14. Optimal Continuation Allocation

Consider:

$$\min_{M_1,\dots,M_K} \sum_j \frac{c_j}{M_j}$$

subject to:

$$\sum_j M_j = B.$$

The Lagrangian is:

$$\mathcal{L} = \sum_j \frac{c_j}{M_j} + \lambda \left(\sum_j M_j - B \right).$$

Setting derivatives to zero:

$$-\frac{c_j}{M_j^2} + \lambda = 0.$$

Thus:

$$M_j = \sqrt{\frac{c_j}{\lambda}}.$$

After normalizing to the budget:

$$\boxed{M_j^* = B \frac{\sqrt{c_j}}{\sum_l \sqrt{c_l}}}$$

or equivalently:

$$\boxed{M_j^* \propto \sigma_j \|J_{:j}\|.}$$

---

15. Integer Greedy Allocator

Since M_j must be an integer, define:

$$E_j(M_j) = \frac{c_j}{M_j}.$$

The reduction from one additional sample is:

$$\Delta_j = \frac{c_j}{M_j} - \frac{c_j}{M_j + 1} = \boxed{\frac{c_j}{M_j(M_j + 1)}}.$$

Implementation:

```
def allocate(c, budget, min_per_thought=1):
    K = len(c)
    M = [min_per_thought] * K
    remaining = budget - sum(M)

    for _ in range(remaining):
        delta = [
            c[j] / (M[j] * (M[j] + 1))
            for j in range(K)
        ]

        j_star = max(
            range(K),
            key=lambda j: delta[j]
        )

        M[j_star] += 1

    return M
```

---

16. Worked Allocation Example

Suppose:

$$c = [1.5,\ 6,\ 6,\ 1.5].$$

Let:

$$B_{main} = 8.$$

Start with one main sample per thought:

$$M = [1, 1, 1, 1].$$

Four samples remain.

Round 1

$$\Delta = [0.75,\ 3,\ 3,\ 0.75].$$

Assign to T_2:

$$M = [1, 2, 1, 1].$$

Round 2

$$\Delta_2 = \frac{6}{2(3)} = 1,$$

while:

$$\Delta_3 = 3.$$

Assign to T_3:

$$M = [1, 2, 2, 1].$$

Round 3

Assign another sample to one of the high-value thoughts:

$$M = [1, 3, 2, 1].$$

Round 4

$$\Delta_2 = \frac{6}{3(4)} = 0.5, \qquad \Delta_3 = \frac{6}{2(3)} = 1.$$

Assign to T_3:

$$\boxed{M = [1, 3, 3, 1].}$$

Uniform main allocation would be:

$$[2, 2, 2, 2].$$

---

17. Main Sampling

Given:

$$M = [1, 3, 3, 1],$$

generate independent main continuations:

```
T1 → 1 new answer
T2 → 3 new answers
T3 → 3 new answers
T4 → 1 new answer
```

Pilot and main samples must come from the same policy snapshot:

```
pilot
→ allocation
→ main
→ compute loss
→ optimizer step
```

The following is invalid:

```
pilot
→ optimizer step
→ main
```

because the sampling distributions would no longer satisfy the assumptions used by the estimator analysis.

---

18. Bias-Controlled Pilot Reuse

A naive sample-size-weighted estimator is:

$$\widehat{\mu}_j^{naive} = \frac{m_0 \bar{R}_j^P + M_j \bar{R}_j^M}{m_0 + M_j}.$$

However:

$$M_j = g(R^P),$$

so the pilot coefficient depends indirectly on the pilot outcome itself.

The initial JADA-GRPO estimator uses a fixed mixing coefficient:

$$\boxed{\widetilde{\mu}_j = \lambda \bar{R}_j^P + (1-\lambda)\bar{R}_j^M}$$

with:

$$0 < \lambda < 1.$$

The key requirement is that λ is fixed before observing the current pilot outcomes.

For the MVP:

$$\lambda = 0.5.$$

Example:

$$\bar{R}_2^P = 0.5, \qquad \bar{R}_2^M = \frac{2}{3}.$$

Then:

$$\tilde{\mu}_2 = 0.5(0.5) + 0.5(0.667) = 0.5835.$$

---

19. Unbiasedness at the Thought-Value Level

The pilot mean satisfies:

$$E[\bar{R}_j^P] = \mu_j.$$

Although M_j depends on the pilot observations, conditional on pilot history $\mathcal{F}_P$, the main samples are generated independently from the same policy snapshot. Therefore:

$$E[\bar{R}_j^M | \mathcal{F}_P] = \mu_j$$

provided that every thought receives at least one main sample.

Hence:

$$E[\tilde{\mu}_j] = \lambda E[\bar{R}_j^P] + (1-\lambda)E[\bar{R}_j^M]$$

$$= \lambda\mu_j + (1-\lambda)\mu_j$$

and therefore:

$$\boxed{E[\tilde{\mu}_j] = \mu_j.}$$

This establishes unbiasedness at the thought-value level.

It does **not** imply:

$$E[f(\tilde{\mu})] = f(\mu),$$

because group normalization is nonlinear. The proposal therefore does not claim an exactly unbiased normalized-advantage estimator. Bias at the normalized-advantage level must be analyzed through delta-method approximations and direct empirical evaluation.

---

20. Final Thought Advantage

From:

$$\tilde{\mu} = [\tilde{\mu}_1, \dots, \tilde{\mu}_K],$$

compute:

$$\bar{\tilde{\mu}} = \frac{1}{K}\sum_j \tilde{\mu}_j,$$

$$s_{\tilde{\mu}} = \sqrt{\frac{1}{K}\sum_j (\tilde{\mu}_j - \bar{\tilde{\mu}})^2 + \epsilon},$$

and:

$$\boxed{\tilde{A}_j = \frac{\tilde{\mu}_j - \bar{\tilde{\mu}}}{s_{\tilde{\mu}}}.}$$

---

21. Answer-Level Loss Under Variable Allocation

When:

$$M_1 \neq M_2 \neq \cdots \neq M_K,$$

simply averaging all answer losses would implicitly give more total loss weight to thoughts receiving more samples.

To preserve equal total weight per thought:

$$\boxed{L_{answer} = \frac{1}{K}\sum_{i=1}^{K}\left[\frac{1}{M_i}\sum_{m=1}^{M_i}\ell_{im}\right].}$$

For:

$$M = [1, 3, 3, 1],$$

we obtain:

```
T1: 1 answer × 1/4      = 1/4 total weight
T2: 3 answers × 1/12    = 1/4 total weight
T3: 3 answers × 1/12    = 1/4 total weight
T4: 1 answer × 1/4      = 1/4 total weight
```

Thus adaptive sampling does not silently change the optimization objective.

---

22. Connection to Policy-Gradient Estimation Error

Let the ideal thought policy gradient be:

$$g = \frac{1}{K}\sum_{i=1}^{K} A_i s_i,$$

where:

$$s_i = \nabla_\theta \log \pi_\theta(T_i|x).$$

The estimated gradient is:

$$\hat{g} = \frac{1}{K} \sum_i \hat{A}_i s_i.$$

Therefore:

$$\hat{g} - g = \frac{1}{K} \sum_i (\hat{A}_i - A_i) s_i.$$

If:

$$\|s_i\|_2 \leq G,$$

then a Cauchy-Schwarz argument yields a bound of the form:

$$\boxed{\|\hat{g} - g\|_2^2 \leq C_G \|\hat{\mathbf{A}} - \mathbf{A}\|_2^2}$$

for an appropriate constant C_G depending on G and K.

The theoretical chain is therefore:

```
better rollout allocation
        ↓
lower advantage-estimation error
        ↓
smaller upper bound on policy-gradient estimation error
        ↓
potentially more stable optimization
```

The theoretical claim should remain limited to gradient-estimation error under stated assumptions. Improved final task performance must be demonstrated empirically.

---

23. Optional Convergence Analysis

Under standard smoothness assumptions and bounded stochastic-gradient variance, a convergence result may take the form:

$$\frac{1}{T} \sum_{t=1}^{T} E\|\nabla F(\theta_t)\|^2 \leq O\left(\frac{F(\theta_0) - F^*}{\eta T} + \eta V_g \right),$$

where:

$$V_g = E\|\hat{g} - g\|^2.$$

JADA-GRPO aims to reduce the component of V_g attributable to noisy thought-advantage estimation.

This analysis should be added only after the estimator-level derivation is fully validated.

---

24. Main Contributions

Contribution 1 — Group-Normalization-Aware Allocation

Derive:

$$\boxed{\mathcal{E}_A \approx \sum_j \frac{\sigma_j^2 \|J_{:j}\|^2}{M_j}.}$$

Unlike variance-only allocation, the criterion captures cross-thought coupling introduced by group normalization.

Contribution 2 — Adaptive Continuation Budget Optimization

Derive:

$$\boxed{M_j^* \propto \sigma_j \|J_{:j}\|.}$$

Implement the discrete allocation using:

$$\boxed{\Delta_j = \frac{c_j}{M_j(M_j+1)}.}$$

Contribution 3 — Bias-Controlled Pilot Reuse and Optimization Link

Reuse pilot data through:

$$\boxed{\tilde{\mu}_j = \lambda \bar{R}_j^P + (1-\lambda)\bar{R}_j^M}$$

with a precommitted λ, and connect advantage-estimation error to policy-gradient estimation error.

---

25. Positioning

25.1. GRPO

GRPO uses group-relative rewards to construct relative policy updates without requiring a separately learned critic.

JADA-GRPO does not replace the underlying GRPO objective. It improves how continuation-generation compute is allocated when estimating thought-level credit.

25.2. GRPO-MA

GRPO-MA uses multiple answer continuations per thought to improve the reliability of thought-value and thought-advantage estimation.

JADA-GRPO asks:

> Under a fixed continuation budget, should every thought receive the same number of answer continuations?

25.3. Variance-Guided Adaptive Rollout

Variance-guided methods allocate additional samples to trajectories or groups with high reward uncertainty.

JADA-GRPO instead uses:

$$\text{uncertainty} \times \text{group-normalization influence}.$$

The distinction is:

$$\sigma_j^2$$

versus:

$$\sigma_j^2 \|J_{:j}\|^2.$$

25.4. Gradient-Aware Tree Branching

Recent tree-branch allocation methods frame branching as compute allocation for reducing policy-gradient estimation error.

JADA-GRPO should therefore be positioned narrowly and precisely as adaptive continuation allocation inside GRPO-MA-style thought groups, where group normalization introduces explicit cross-thought credit coupling.

The paper should not claim to be the first adaptive branching or adaptive rollout-allocation method.

---

26. Experimental Strategy

The experiments should proceed in stages:

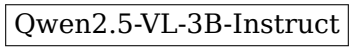
```
Phase A
Synthetic estimator validation
        ↓
Phase B
Frozen-model reference-pool evaluation
        ↓
Phase C
Gradient-error evaluation
        ↓
Phase D
Small-scale online RL training
        ↓
Phase E
Full benchmark and ablation study
```

The method should not proceed directly to expensive online RL training before the estimator-level hypotheses are validated.

---

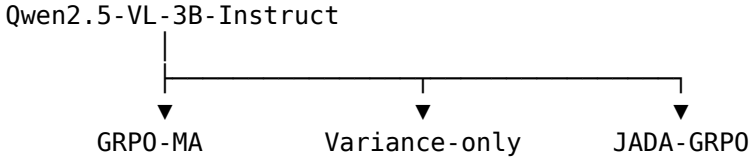
27. Starting Checkpoint and SFT Policy

The initial experiment should start from:



without an additional SFT stage.

This keeps the comparison against GRPO-MA controlled:



Before RL training, run a pre-RL sanity evaluation on approximately 200–500 training prompts and measure:

1. output-format compliance;
2. average reward;
3. reward sparsity;
4. per-thought reward variance;
5. heterogeneity across thoughts.

SFT should be introduced only if the starting model has one of the following problems:

Case A — Severe Format Failure

The model can solve tasks but regularly violates the output format required by the reward parser.

Case B — Reward Is Almost Always Zero

If most groups look like:

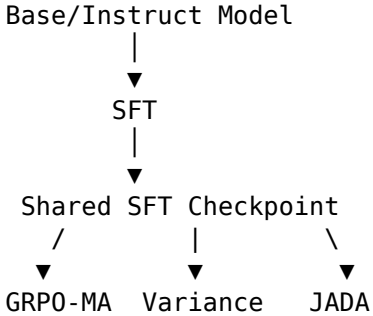
```
T1 → 0 0 0 0
T2 → 0 0 0 0
T3 → 0 0 0 0
T4 → 0 0 0 0
```

then both GRPO-MA and JADA-GRPO have little useful learning or allocation signal.

Case C — Raw Base Model Instead of an Instruction-Tuned Checkpoint

A short SFT warm start may be appropriate before RL.

If SFT is required, it must be performed once, producing a common checkpoint:



All compared RL methods must start from the same SFT checkpoint.

---

28. Model Selection

Primary Model

Qwen2.5-VL-3B-Instruct

Purpose:

- direct comparison with the GRPO-MA setup;
- lower training cost;
- sufficient capacity for repeated estimator and RL experiments.

Scaling Model

Qwen2.5-VL-7B-Instruct

Purpose:

- test whether the allocation effect persists at a larger model scale.

Generalization Model

Qwen3-4B

Purpose:

- test whether the method generalizes beyond the exact model family and implementation used by the main GRPO-MA comparison.

Recommended sequence:

```
3B detailed study
      ↓
7B scaling check
      ↓
Qwen3-4B family generalization
```

---

29. Experiment A — Synthetic Validation

Use true success probabilities:

$$p = [0.95,\ 0.55,\ 0.45,\ 0.05].$$

Generate:

$$R_{jm} \sim \text{Bernoulli}(p_j).$$

The true advantage is:

$$A^* = f(p).$$

Each Monte Carlo trial performs:

```
1. sample pilot observations
2. estimate uncertainty
3. compute Jacobian
4. allocate fixed budget
5. sample main observations
6. estimate final thought values
7. compute estimated advantage
8. compare with A*
```

Run approximately:

```
10,000–100,000 trials
```

This experiment requires no GPU.

---

30. Synthetic Baselines

B1 — Uniform

$$M_j = B/K.$$

B2 — Variance-Only

$$M_j \propto \hat{\sigma}_j.$$

B3 — Jacobian-Only

$$M_j \propto \|\hat{J}_{:j}\|.$$

B4 — JADA-GRPO

$$M_j \propto \hat{\sigma}_j \|\hat{J}_{:j}\|.$$

B5 — Oracle

Use true p_j values to construct the true variance and Jacobian information.

The oracle is not a practical baseline; it estimates the ceiling under perfect knowledge.

---

31. Synthetic Metrics

Advantage MSE

$$\boxed{MSE_A = \frac{1}{K}\sum_i (\hat{A}_i - A_i^*)^2.}$$

Bias

$$Bias_i = E[\hat{A}_i] - A_i^*.$$

Sign Error Rate

$$\boxed{P\left(\text{sign}(\hat{A}_i) \neq \text{sign}(A_i^*)\right).}$$

Rank Correlation

$$Spearman(\hat{\mathbf{A}}, \mathbf{A}^*).$$

---

32. Go/No-Go Gate 1

Proceed to frozen-model experiments only if Jacobian-aware allocation consistently outperforms variance-only allocation across multiple probability configurations and budgets.

A strong result would look like:

```
Advantage MSE
Uniform          1.00
Variance-only    0.82
JADA             0.60
```

A weak result such as:

```
Uniform          1.00
Variance-only    0.82
JADA             0.80
```

would indicate that the Jacobian contribution is too small to justify expensive RL experiments without further method development.

---

33. Experiment B — Frozen-Model Reference Pool

Freeze the model parameters.

For each prompt:

1. generate K thoughts;
2. generate a large continuation pool for each thought.

Use:

$$N_{ref} = 128$$

or:

$$N_{ref} = 256$$

if compute allows.

Reference thought value:

$$\mu_j^* = \frac{1}{N_{ref}} \sum_{m=1}^{N_{ref}} R_{jm}.$$

Reference advantage:

$$A^* = f(\mu^*).$$

Smaller rollout budgets can then be simulated by resampling from the stored pool.

Example:

```
T2 reference pool:
128 answers
73 correct
55 incorrect
```

Then:

$$\mu_2^* = \frac{73}{128} = 0.5703.$$

For each simulated budget:

```
sample pilot
→ compute allocation
→ sample additional answers
→ compute estimated advantage
→ compare with A*
```

This isolates estimator quality from RL optimization dynamics.

---

34. Experiment C — Pilot-Reuse Bias

Compare:

C1 — No Reuse

Pilot samples affect allocation but not the final value estimate.

C2 — Naive Sample-Size-Weighted Reuse

$$\hat{\mu}_j = \frac{m_0 \bar{R}_j^P + M_j \bar{R}_j^M}{m_0 + M_j}.$$

C3 — Fixed-Weight Reuse

$$\tilde{\mu}_j = \lambda \bar{R}_j^P + (1-\lambda)\bar{R}_j^M.$$

Measure:

$$Bias(\hat{\mu}_j)$$

and:

$$Bias(\hat{A}_j).$$

Expected trade-off:

```
No reuse
→ minimal adaptive-selection interaction
→ wastes pilot information

Naive reuse
→ potentially lower variance
→ may introduce adaptive-selection bias

Fixed-weight reuse
→ uses pilot information
→ controls direct μ-level adaptive-selection bias
```

---

35. Experiment D — Gradient Error

Construct a high-quality reference gradient using the reference advantage:

$$g^* = \frac{1}{K} \sum_i A_i^* \nabla_\theta \log \pi_\theta(T_i|x).$$

For each allocation method, compute:

$$\hat{g}.$$

Primary metrics:

Gradient MSE

$$\boxed{\|\hat{g} - g^*\|_2^2.}$$

Gradient Cosine Similarity

$$\boxed{\cos(\hat{g}, g^*).}$$

Full-model gradient comparison may be expensive. Practical alternatives include:

- LoRA-only gradient evaluation;
- selected transformer blocks;
- random low-dimensional projections;
- streaming dot-product computation without storing the full vector.

The initial implementation should use LoRA parameters.

---

36. Go/No-Go Gate 2

Proceed to large online training if:

1. JADA reduces advantage MSE relative to variance-only allocation;
2. JADA improves gradient agreement with the reference gradient;
3. pilot reuse does not introduce substantial estimator bias;
4. allocation and Jacobian overhead remain small relative to generation cost.

---

37. Experiment E — Online RL Training

Start with:

```
1 model
1 task
3 methods
3 random seeds
```

Methods:

```
GRPO-MA Uniform
Variance-only Adaptive
JADA-GRPO
```

Only after a clear signal is observed should the study expand to additional baselines, tasks, and model scales.

---

38. Online Training Metrics

Task Performance

Examples:

```
Accuracy
Pass@1
Average reward
```

Compute Efficiency

Measure:

```
rollouts to target performance
generated tokens
GPU-hours
wall-clock time
```

Training Stability

Measure:

```
gradient norm spikes
reward variance
advantage variance
KL divergence
training collapse rate
```

Allocation Dynamics

Track:

```
pilot variance
Jacobian influence
allocation entropy
distribution of Mj
fraction of near-uniform groups
```

---

39. Same-Rollout vs Same-Token Evaluation

Equal continuation counts do not necessarily imply equal generation cost because continuation lengths can differ.

Therefore, evaluate two settings.

Setting A — Same Continuation Count

Fix:

$$B_{answers}.$$

Setting B — Same Generated-Token Budget

Fix or approximately match:

$$B_{tokens}.$$

Compute-efficiency claims should be based primarily on generated tokens, wall-clock time, and GPU-hours rather than only the number of sampled answers.

---

40. Core Ablations

A1 — Remove Jacobian

$$c_j = \sigma_j^2.$$

Question:

> Is the group-normalization sensitivity term necessary?

A2 — Remove Variance

$$c_j = \|J_{:j}\|^2.$$

A3 — Uniform Allocation

No adaptive allocation.

A4 — Pilot Size

$$m_0 \in \{1, 2, 4\}.$$

A5 — Total Budget

$$B \in \{16, 32, 64\}.$$

A6 — Pilot Reuse

```
No reuse
Naive reuse
Fixed-weight reuse
```

A7 — Reuse Coefficient

$$\lambda \in \{0.25, 0.5, 0.75\}.$$

A8 — Jacobian Computation

```
full autograd Jacobian
analytic Jacobian
diagonal approximation
```

---

41. Expected Main Figures

Figure 1 — Method Overview

```
Prompt
→ Thoughts
→ Pilot
→ uncertainty + Jacobian
→ adaptive allocation
→ main samples
→ bias-controlled reuse
→ normalized advantage
→ GRPO update
```

Figure 2 — Advantage MSE vs Budget

```
y-axis: advantage MSE
x-axis: rollout budget

Uniform
Variance-only
JADA-GRPO
Oracle
```

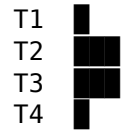
Figure 3 — Gradient Error vs Budget

```
y-axis: ||ĝ - g*||²
x-axis: rollout budget
```

Figure 4 — Performance vs Generated Tokens

```
y-axis: task performance
x-axis: cumulative generated tokens
```

Figure 5 — Allocation Visualization



shown alongside reward variance, Jacobian influence, and allocated continuation count.

---

42. Implementation Architecture

```
project/
│
├── trainer/
│   ├── grpo_ma_trainer.py
│   ├── jada_trainer.py
│   │
│   ├── allocation/
│   │   ├── pilot_stats.py
│   │   ├── jacobian.py
│   │   ├── allocator.py
│   │   └── reuse_estimator.py
│   │
│   ├── advantage/
│   │   ├── thought_advantage.py
│   │   └── answer_advantage.py
│   │
│   └── losses/
│       └── weighted_grpo_loss.py
│
├── experiments/
│   ├── synthetic/
│   │   ├── simulate_bernoulli.py
│   │   └── plot_advantage_mse.py
│   │
│   ├── frozen_pool/
│   │   ├── build_reference_pool.py
│   │   ├── simulate_allocation.py
│   │   └── evaluate_estimators.py
│   │
│   ├── gradient/
│   │   └── evaluate_gradient_error.py
│   │
│   └── online/
│       └── train.py
│
├── configs/
│   ├── uniform.yaml
│   ├── variance.yaml
│   └── jada.yaml
│
└── tests/
    ├── test_jacobian.py
    ├── test_allocator.py
    ├── test_reuse_unbiased.py
    └── test_weighted_loss.py
```

---

43. Core APIs

Pilot Statistics

```python
def estimate_pilot_stats(
    rewards,
```

```
    alpha0=0.5,
    beta0=0.5,
):
    """
    rewards:
        Tensor [K, m0]

    returns:
        mu_hat: [K]
        var_hat: [K]
    """
```

Jacobian

```
def compute_advantage_jacobian(
    mu_hat,
    eps=1e-6,
):
    """
    returns:
        J: [K, K]
    """
```

Information Score

```
def compute_information_score(
    var_hat,
    J,
):
    influence = (J ** 2).sum(dim=0)
    return var_hat * influence
```

Allocator

```
def allocate_budget(
    c,
    total_budget,
    min_per_thought=1,
):
    ...
```

Pilot-Reuse Estimator

```
def fixed_weight_reuse(
    pilot_mean,
    main_mean,
    reuse_lambda=0.5,
):
    return (
        reuse_lambda * pilot_mean
        +
        (1 - reuse_lambda) * main_mean
    )
```

---

44. End-to-End Pseudocode

```
def jada_rollout(prompt, policy, K, m0, B_main):

    # 1. Generate thoughts
    thoughts = generate_thoughts(
        policy=policy,
        prompt=prompt,
        num_thoughts=K,
    )

    # 2. Pilot sampling
    pilot_rewards = []

    for thought in thoughts:
        answers = generate_answers(
            policy=policy,
            thought=thought,
            num_answers=m0,
        )

        rewards = reward_fn(answers)
        pilot_rewards.append(rewards)

    # 3. Pilot statistics
    mu_pilot, var_pilot = estimate_pilot_stats(
        pilot_rewards
    )

    # 4. Group-normalization sensitivity
    J = compute_advantage_jacobian(
        mu_pilot
    )

    influence = (J ** 2).sum(dim=0)
    c = var_pilot * influence

    # 5. Allocate main budget
    M = allocate_budget(
        c=c,
        total_budget=B_main,
        min_per_thought=1,
    )

    # 6. Main sampling
    main_answers = []
    main_rewards = []

    for thought, m in zip(thoughts, M):
        answers = generate_answers(
            policy=policy,
            thought=thought,
            num_answers=m,
        )

        rewards = reward_fn(answers)
```

```
            main_answers.append(answers)
            main_rewards.append(rewards)

        # 7. Bias-controlled pilot reuse
        pilot_mean = mean_per_thought(
            pilot_rewards
        )

        main_mean = mean_per_thought(
            main_rewards
        )

        mu_final = fixed_weight_reuse(
            pilot_mean,
            main_mean,
            reuse_lambda=0.5,
        )

        # 8. Thought advantage
        thought_adv = group_advantage(
            mu_final
        )

        # 9. Weighted GRPO loss
        loss = compute_weighted_grpo_loss(
            thoughts=thoughts,
            main_answers=main_answers,
            thought_advantage=thought_adv,
            main_rewards=main_rewards,
            allocation=M,
        )

        return loss
```

---

45. Implementation Invariants

The implementation should enforce:

```
assert len(M) == K
assert sum(M) == B_main
assert min(M) >= 1
```

The policy snapshot used for pilot and main generation must be identical:

```
pilot_policy_version == main_policy_version
```

Loss weighting must satisfy approximately:

```
total_answer_weight_per_thought == 1 / K
```

This ensures that variable branching does not silently alter the optimization objective.

---

46. Required Unit Tests

Test 1 — Advantage Normalization

Verify:

$$\sum_i A_i \approx 0.$$

Test 2 — Jacobian Finite-Difference Check

```
torch.allclose(
    J_autograd,
    J_finite_difference,
    atol=...
)
```

Test 3 — Budget Conservation

```
sum(M) == B
```

Test 4 — Marginal Allocation

If one c_j is much larger than all others, that thought should receive more samples.

Test 5 — Symmetry

If:

$$c_1 = c_2 = \cdots = c_K,$$

allocation should be approximately uniform.

Test 6 — Fixed-Weight Reuse Monte Carlo

Verify numerically:

$$E[\tilde{\mu}_j] \approx \mu_j.$$

Test 7 — Weighted Loss

For:

$$M = [1, 3, 3, 1],$$

the total answer-level weight of each thought must be equal.

---

47. Development Timeline

Week 1 — Mathematics and Synthetic Validation

Day 1

- implement `group_advantage`;
- reproduce existing advantage computation;
- add normalization unit tests.

Day 2

- implement Jacobian;
- validate with finite differences.

Day 3

- implement Beta-smoothed pilot statistics;
- implement c_j.

Day 4

- implement integer allocator;
- implement synthetic Monte Carlo simulation.

Day 5

- compare uniform, variance-only, Jacobian-only, and JADA allocation.

Gate: do not proceed to expensive RL training unless the Jacobian-aware method clearly improves over variance-only allocation.

Week 2 — Frozen-Model Evaluation

- build the large reference continuation pool;
- simulate budget allocation from the stored pool;
- evaluate pilot reuse strategies;
- analyze advantage MSE, sign error, bias, rank correlation, and allocation distribution.

Week 3 — Gradient Evaluation

- implement reference-gradient computation;
- implement estimated-gradient computation;
- measure gradient MSE and gradient cosine similarity;
- analyze whether lower advantage error consistently corresponds to lower gradient-estimation error.

Week 4 — Trainer Integration

Implement:

```
pilot generation
→ allocation
→ variable-size main generation
→ pilot reuse
→ normalized advantage
```

```
→ weighted loss
→ optimizer step
```

Variable-size generation is likely to be the main engineering difficulty. The first implementation may use simple loops; after correctness is established, generation can be optimized using buckets grouped by continuation count.

Week 5 — Small Online RL Experiment

Minimum study:

```
3 methods
× 3 seeds
× 1 task
```

Methods:

```
GRPO-MA Uniform
Variance-only Adaptive
JADA-GRPO
```

Week 6+ — Full Study

Extend to:

```
second task
second model scale
additional model family
pilot-size ablation
budget ablation
reuse ablation
lambda ablation
token-matched evaluation
```

---

48. Estimated Research Time

Proof of Concept

2–3 weeks

Expected outputs:

- synthetic validation;
- frozen-pool evaluation;
- first gradient-error analysis;
- basic trainer integration.

Research Prototype

4–6 weeks

Expected outputs:

- estimator evidence;
- gradient evidence;
- online RL results;

- three seeds;
- initial ablations.

Submission-Level Study

6–10 weeks

depending on GPU availability, generation throughput, debugging time, benchmark count, model count, and failed runs.

---

49. Main Risks

Risk 1 — Jacobian Adds Little Beyond Variance

Example:

```
Variance-only MSE = 0.80
JADA MSE          = 0.79
```

This would weaken the central novelty claim.

Mitigation: run synthetic and frozen-model estimator experiments before full RL training.

Risk 2 — Pilot Estimates Are Too Noisy

With:

$$m_0 = 2,$$

both variance and Jacobian estimates may be unstable.

Mitigation: use Beta smoothing, pilot-size ablation, historical priors, Jacobian norm clipping, and fallback rules for degenerate groups.

Risk 3 — Group Standard Deviation Is Near Zero

If:

$$\mu_1 \approx \cdots \approx \mu_K,$$

then:

$$s_\mu \approx 0,$$

which can produce very large Jacobian values.

Mitigation: use:

$$s_\mu = \sqrt{v_\mu + \epsilon}$$

and optionally Jacobian norm clipping or a fallback to variance-only/uniform allocation when group dispersion is below a threshold.

Risk 4 — Pilot-Reuse Estimator Is Not Variance-Optimal

A fixed λ provides a clean bias analysis at the thought-value level but may not minimize variance.

Mitigation: use fixed-weight reuse as the theoretically clean initial version, then investigate historical variance-based weighting, cross-fitting, adaptive-sampling estimators, or martingale-based corrections.

Risk 5 — Two-Stage Generation Increases Wall-Clock Time

Even with the same number of answers, pilot-plus-main generation can introduce synchronization and batching overhead.

Mitigation: report continuation count, generated tokens, wall-clock time, and GPU-hours.

Risk 6 — Variable M_i Complicates Batching

Mitigation: begin with correctness-focused ragged loops, then optimize using continuation-count buckets.

---

50. Fallback Strategies

Case A — Variance-Only Works, Jacobian Does Not

Possible pivot:

`efficient adaptive continuation allocation for GRPO-MA`

The novelty would be substantially weaker.

Case B — Jacobian Reduces Advantage MSE but Does Not Improve Online RL

Investigate:

- mismatch between estimator MSE and training objective;
- gradient weighting;
- policy-likelihood terms;
- token-level effects;
- critic-free normalization behavior.

Case C — Pilot Reuse Is Unstable

Use no-reuse JADA as the main method and move pilot reuse to the ablation/appendix. The core contribution remains:

$$\sigma_j^2 \|J_{:j}\|^2.$$

---

51. Success Criteria

Minimum Scientific Success

JADA must outperform variance-only allocation in:

$$MSE_A.$$

Stronger Success

Additionally:

$$GradientMSE_{JADA} < GradientMSE_{variance}.$$

Strong Experimental Result

At the same target performance:

$$\boxed{\text{fewer generated tokens and/or GPU-hours}}$$

or, at matched compute:

$$\boxed{\text{higher final task performance.}}$$

The strongest paper would demonstrate both estimator-level improvement and compute-efficient online RL improvement.

---

52. Recommended Paper Narrative

The paper should not be framed as:

> Generate more answers for promising thoughts.

It should also not be framed only as:

> Allocate more continuations to high-variance thoughts.

A stronger formulation is:

> GRPO-MA estimates thought-level credit using multiple answer continuations, but equal continuation counts ignore heterogeneous estimation difficulty. More importantly, group normalization couples all thought advantages, so local reward variance alone does not measure the global value of an additional continuation. We formulate continuation generation as a fixed-budget estimation problem, derive a Jacobian-aware allocation rule that directly targets group-normalized advantage error, reuse pilot rollouts through a bias-controlled estimator, and connect improved advantage estimation to reduced policy-gradient estimation error.

---

53. One-Sentence Method Definition

> **JADA-GRPO allocates continuation rollouts according to each thought's reward uncertainty and Jacobian-measured influence on the entire group-normalized advantage vector, then combines pilot and adaptively sampled rollouts through a bias-controlled estimator before the GRPO update.**

Mathematically:

$$\boxed{c_j = \sigma_j^2 \|J_{:j}\|^2}$$

determines where additional compute is valuable,

$$\boxed{M_j^* \propto \sqrt{c_j}}$$

determines how much compute each thought receives, and:

$$\boxed{\tilde{\mu}_j = \lambda \bar{R}_j^P + (1-\lambda)\bar{R}_j^M}$$

determines how pilot and main samples are combined.

---

54. Immediate Implementation Order

```
[1] Implement group_advantage()
        ↓
[2] Implement Jacobian()
        ↓
[3] Validate Jacobian with finite differences
        ↓
[4] Implement Beta-smoothed pilot statistics
        ↓
[5] Implement cj
        ↓
[6] Implement integer greedy allocator
        ↓
[7] Run synthetic Monte Carlo
        ↓
[8] Compare Uniform / Variance / JADA
        ↓
[9] Implement fixed-weight pilot reuse
        ↓
[10] Measure μ bias and advantage bias
        ↓
[11] Build frozen-model reference pool
        ↓
[12] Evaluate gradient-estimation error
        ↓
[13] Integrate online GRPO training
```

---

55. Initial Configuration

Recommended starting configuration:

```yaml
model:
  name: Qwen2.5-VL-3B-Instruct
  use_sft: false
```

```yaml
thoughts:
  K: 4

pilot:
  answers_per_thought: 2

main:
  total_budget: 8
  min_per_thought: 1

beta_prior:
  alpha: 0.5
  beta: 0.5

reuse:
  lambda: 0.5

advantage:
  eps: 1.0e-6

allocator:
  type: jacobian_variance_greedy
```

Total generated answers:

```
pilot = 4 × 2 = 8
main  = 8
total = 16
```

This can be compared directly with a GRPO-MA configuration using:

```
4 thoughts × 4 answers = 16 answers
```

The comparison must additionally report generated tokens and wall-clock/GPU cost because two-stage generation introduces overhead not reflected by answer count alone.

---

56. Recommended Evaluation Matrix

Main 3B Study

```
Qwen2.5-VL-3B-Instruct

1. GRPO-MA Uniform
2. Variance-only Adaptive
3. JADA without pilot reuse
4. JADA with bias-controlled pilot reuse

× 3 seeds
```

Scaling Study

```
Qwen2.5-VL-7B-Instruct

1. GRPO-MA Uniform
2. JADA
```

```
× 3 seeds
```

Generalization Study

```
Qwen3-4B

1. GRPO-MA-style Uniform
2. JADA

× 3 seeds
```

---

57. Primary Hypotheses

H1 — Advantage Estimation

$$\boxed{MSE_A^{JADA} < MSE_A^{GRPO-MA}}$$

under a fixed continuation budget.

H2 — Gradient Estimation

$$\boxed{GradientError^{JADA} < GradientError^{GRPO-MA}.}$$

H3 — Compute Efficiency

$$\boxed{ComputeToTarget^{JADA} < ComputeToTarget^{GRPO-MA}.}$$

H4 — Final Performance

$$\boxed{FinalPerformance^{JADA} \geq FinalPerformance^{GRPO-MA}}$$

at matched compute, with improvement preferred but not required for the main efficiency claim.

---

58. Expected Outcome Pattern

The strongest expected result is not necessarily a very large absolute increase in final benchmark accuracy.

A convincing result could be:

```
Target accuracy: 52%

GRPO-MA:
100M generated tokens
12 GPU-hours

JADA-GRPO:
72M generated tokens
9 GPU-hours
```

Alternatively, at a matched generation budget:

```
100M generated tokens

GRPO-MA: 52.0%
JADA:    53.5%
```

The central objective is therefore:

better credit estimation per unit of generation compute

rather than maximizing final accuracy regardless of cost.

---

59. Key References

1. **DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models**
   Introduces Group Relative Policy Optimization (GRPO).
   https://arxiv.org/abs/2402.03300
2. **GRPO-MA: Multi-Answer Generation in GRPO for Stable and Efficient Chain-of-Thought Training**
   Establishes multi-answer thought evaluation and analyzes thought-advantage variance.
   https://arxiv.org/abs/2509.24494
3. **Learning as Reasoning Unfolds: Progressive Rollout Allocation for Efficient Reinforcement Learning (VIGOR)**
   Uses reward variance to progressively allocate rollout budget.
   https://arxiv.org/abs/2607.22002
4. **EPIG-Tree: Compute-Optimal Branching for Gradient-Efficient Reinforcement Learning**
   Frames adaptive branching as compute allocation for policy-gradient estimation.
   https://arxiv.org/abs/2609.20004

---

60. Final Research Hypotheses

The primary hypothesis is:

> **H1.** Under a fixed continuation budget, allocating rollouts according to $\sigma_j^2 \|J_{:j}\|^2$ yields lower group-normalized thought-advantage estimation error than uniform or variance-only allocation.

The optimization hypothesis is:

> **H2.** Lower thought-advantage estimation error reduces policy-gradient estimation error and improves RL performance per unit of generation compute.

The pilot-reuse hypothesis is:

> **H3.** A precommitted fixed-weight combination of pilot and post-allocation samples reuses pilot compute while avoiding direct adaptive-selection bias at the thought-value estimation level.

If these hypotheses are supported, JADA-GRPO forms a unified method combining:

```
group-aware compute allocation
+
efficient reuse of collected rollouts
+
a theoretical link to policy-gradient estimation quality
```

rather than a heuristic modification of the number of answer continuations per thought.